

BÁO CÁO THIẾT KẾ VÀ VẬN HÀNH BACKEND

LifeBalance - Multi-module Spring Boot Microservices

Phạm vi	Tất cả 12 module trong thư mục lifebalance-backend
Mục tiêu	Giải thích cách thiết kế, cách vận hành, cách viết code và công dụng từng module
Kiến trúc	Microservice backend, Spring Boot 3.5.x, Spring Cloud, Eureka, Gateway, PostgreSQL, Keycloak
Ngày tạo	25/08/2026
File đầu ra	E:\lifebalance-be\lifebalance-backend\Bao_cao_LifeBalance_Backend_Modules.docx

Tóm tắt nhanh

Backend được chia thành 4 nhóm chính: hạ tầng điều phối/truy cập, thư viện dùng chung, service nghiệp vụ lõi và service phân tích/hỗ trợ. Gateway là cửa vào hệ thống, discovery-server là danh bạ service, các service nghiệp vụ tự quản lý domain và database riêng.

Mục lục nội dung

- 1. Tổng quan kiến trúc backend
- 2. Cách vận hành hệ thống
- 3. Cách viết code trong backend
- 4. Bảng tổng hợp tất cả module
- 5. Phân tích chi tiết từng module
- 6. Checklist review và bảo vệ đồ án
- 7. Rủi ro, tác động và khuyến nghị
- 8. Nguồn tham chiếu trong repo

1. Tổng quan kiến trúc backend

LifeBalance backend được xây dựng dưới dạng multi-module Maven project, dùng Java 21, Spring Boot 3.5.x và Spring Cloud. Cách chia module hướng tới microservice: mỗi bounded context có service riêng, database riêng, migration riêng và API riêng; các concern dùng chung được đưa vào module thư viện.

Luồng request chính

Tầng	Module/thành phần	Vai trò
Client/Frontend	Web/mobile/client API	Gửi request đến gateway qua HTTP.
Gateway	gateway	Xác thực ở biên, định tuyến theo path, resolve service qua Eureka.
Service Registry	discovery-server	Nhận đăng ký instance và cung cấp danh sách service runtime.
Domain Services	identity, task, timeline, resource-capital, finance, notification, analytics, ai	Xử lý nghiệp vụ theo từng bounded context.
Shared Libraries	lifebalance-common, lifebalance-security	Chuẩn hóa response/error/security để các service nhất quán.
Infrastructure	PostgreSQL, Keycloak, optional RabbitMQ/Redis/MinIO/Prometheus/Grafana	Lưu dữ liệu, định danh, message/cache/storage và quan sát hệ thống.
Ý đồ thiết kế Hệ thống tách module theo nghiệp vụ để dễ scale, test và review. Gateway/discovery xử lý điều phối; common/security xử lý cross-cutting; service nghiệp vụ chỉ tập trung domain của mình.		

Backend gồm 4 nhóm chính

Nhóm	Module	Mục đích
Hạ tầng	discovery-server, gateway	Service registry và API endpoint.
Dùng chung	lifebalance-common, lifebalance-security	Contract response/error và bảo mật JWT/OAuth2.
Nghiệp vụ lõi	identity, task, timeline, resource-capital, finance	Các bounded context trực tiếp tạo giá trị nghiệp vụ.
Hỗ trợ/phân tích	notification, analytics, ai	Thông báo, báo cáo, insight và recommendation.

2. Cách vận hành hệ thống

Backend được vận hành chủ yếu bằng Docker Compose. Các lệnh dưới đây chạy tại thư mục lifebalance-backend.

Mục đích	Lệnh/URL
Tạo file môi trường	Copy-Item .env.example .env
Build và chạy hệ thống chính	docker compose up -d --build
Kiểm tra container	docker compose ps
Xem log gateway	docker compose logs -f gateway
Health gateway	http://localhost:8080/actuator/health/readiness
Eureka UI	http://localhost:8761
Keycloak	http://localhost:8082
Dừng hệ thống	docker compose down

Port quan trọng

Thành phần	Port/URL	Ghi chú
Gateway	localhost:8080	Điểm vào public cho frontend/client.
Eureka	localhost:8761	Xem service đã đăng ký.
Keycloak	localhost:8082	Issuer OAuth2/JWT và quản lý realm.
PostgreSQL	localhost:5432	Database khi cần kiểm tra trực tiếp.
Prometheus	localhost:9090	Bật bằng profile monitoring.
Grafana	localhost:3000	Bật bằng profile monitoring.

Quy tắc runtime trong Docker

- Trong container không dùng localhost để gọi service khác; phải dùng Docker DNS như postgres:5432, keycloak:8080, discovery-server:8080 và gateway:8080.
- Gateway là service public ở edge network; phần lớn service nghiệp vụ chỉ cần internal network.
- Các service expose health/readiness/liveness để Compose và monitoring đánh giá trạng thái.
- Optional profiles messaging/cache/storage/monitoring bật thêm RabbitMQ, Redis, MinIO, Prometheus và Grafana.

3. Cách viết code trong backend

Code backend đi theo pattern quen thuộc của Spring Boot: controller nhận request, service xử lý nghiệp vụ, repository truy cập dữ liệu, DTO làm contract API và validation bảo vệ input.

Layer/package	Công dụng	Quy ước review
controller	Expose REST endpoint, nhận request, gọi service.	Không đặt nghiệp vụ phức tạp trong controller.
dto	Request/response object, validation annotation.	Không expose entity trực tiếp ra API.
service	Business rule, transaction boundary, integration orchestration.	Đây là nơi review logic nghiệp vụ chính.
repository	JPA query/persistence.	Không đặt rule nghiệp vụ vào repository.
model/domain	Entity/value/domain model.	Giữ invariant và quan hệ dữ liệu rõ ràng.
error/exception	Mã lỗi, exception, global handler.	Lỗi trả về phải nhất quán qua ApiResponse/ApiError.
integration/infrastructure	Adapter tới service khác hoặc hệ thống ngoài.	Tách khỏi domain để dễ thay đổi provider.
test	Unit/integration tests.	Ưu tiên test rule nghiệp vụ, security, route và migration.

Luồng xử lý request tiêu chuẩn

- Client gọi Gateway qua /api/...; gateway xác thực và route tới service đích bằng lb://SERVICE-NAME.
- Controller của service nhận request, validate DTO và lấy thông tin user từ security context.
- Service kiểm tra quyền sở hữu dữ liệu, áp dụng business rule và gọi repository/integration khi cần.
- Repository đọc/ghi database riêng của service.
- Kết quả trả về theo ApiResponse; lỗi trả về theo ApiError để frontend xử lý thống nhất.

Nguyên tắc viết code nên giữ

- Mỗi module chỉ xử lý domain của mình; không import trực tiếp entity của module nghiệp vụ khác.
- Không hard-code URL service; dùng gateway, Eureka, env hoặc configuration property.
- Bảo vệ dữ liệu theo userId/owner trong service, không chỉ dựa vào frontend.
- Các thay đổi dữ liệu quan trọng cần có transaction và test case.
- Nếu thêm endpoint mới: cập nhật controller, DTO, service, test, gateway route và tài liệu API.

4. Bảng tổng hợp tất cả module

Module	Nhóm	Vai trò	Quy mô	Nhận xét nhanh
discovery-server	Hạ tầng điều phối	Service Registry	1 Java / 2 test	Điểm mạnh: module nhỏ, dễ kiểm soát, đúng vai trò hạ tầng.
lifebalance-common	Thư viện dùng chung	Shared API/Error/Web Utilities	10 Java / 2 test	Điểm mạnh: giảm trùng lặp và giữ format lỗi nhất quán.
lifebalance-security	Thư viện dùng chung	Security Auto-Configuration	13 Java / 9 test	Điểm mạnh: security tập trung, dễ review hơn so với cấu hình rải rác.
gateway	Hạ tầng truy cập	API Gateway	1 Java / 3 test	Điểm mạnh: gateway mỏng, đúng vai trò định tuyến, dễ giải thích khi bảo vệ.
identity-service	Nghiệp vụ lõi	Identity, Authorization, Administration Support	179 Java / 43 test	Điểm mạnh: test nhiều nhất trong backend, phù hợp vì identity là module rủi ro cao.
task-service	Nghiệp vụ lõi	Task Management	89 Java / 19 test	Điểm mạnh: bounded context rõ, số lượng test tương đối tốt.
timeline-service	Nghiệp vụ lõi	Timeline and Scheduling	40 Java / 4 test	Điểm mạnh: module tách riêng scheduling nên task-service không bị phình logic thời gian.
resource-capital-service	Nghiệp vụ lõi	Resource Capital Management	145 Java / 47 test	Điểm mạnh: số lượng test cao, phù hợp với module có nhiều rule nghiệp vụ.
finance-service	Nghiệp vụ lõi	Finance Management	74 Java / 5 test	Điểm mạnh: bounded context tài chính rõ ràng.
notification-service	Nghiệp vụ hỗ trợ	Notification and Preference Management	57 Java / 4 test	Điểm mạnh: notification tách riêng nên task/finance không phải gánh logic gửi.
analytics-service	Nghiệp vụ phân tích	Tracking, Evaluation, Reporting	61 Java / 8 test	Điểm mạnh: tracking/evaluation được tách khỏi module tạo dữ liệu.
ai-service	Nghiệp vụ phân tích	AI Recommendations and Insights	55 Java / 5 test	Điểm mạnh: AI được tách riêng nên không làm nhiễm service nghiệp vụ core.

Cách đọc bảng

Module có Java/test count cao thường chứa nhiều nghiệp vụ hoặc nhiều rule cần kiểm thử. Module hạ tầng như gateway/discovery có ít code Java vì phần lớn hành vi nằm trong cấu hình Spring Cloud.

5. Phân tích chi tiết từng module

5.1. Module discovery-server

Vai trò	Service Registry
Entrypoint	com.lifebalance.discovery.DiscoveryServerApplication
Số file Java/test	1 Java, 2 test
Dependency chính	Spring Cloud Netflix Eureka Server, Actuator, Prometheus registry, Caffeine
Endpoint/contract chính	/eureka, /actuator/health, /actuator/prometheus
Port/vận hành	8761 trên máy host; 8080 trong Docker network

Công dụng

- Làm danh bạ dịch vụ cho toàn bộ hệ thống microservice.
- Cho phép gateway và các service tìm nhau bằng tên logic thay vì hard-code IP/port.
- Tăng tính linh hoạt khi service được scale, restart hoặc đổi container.

Cách thiết kế

- Ứng dụng Spring Boot rất mỏng, bật vai trò Eureka Server bằng @EnableEurekaServer.
- Không chứa nghiệp vụ; nhiệm vụ chính là nhận đăng ký, gia hạn heartbeat và trả danh sách instance.
- Cấu hình theo profile dev/prod để tách địa chỉ khi chạy local và khi chạy bằng Docker Compose.

Cách viết code trong module

- Giữ class application tối giản, không trộn logic nghiệp vụ vào discovery.
- Cấu hình nên đặt ở application.yaml/application-prod.yaml thay vì viết cố định trong code.
- Test cần xác nhận prod profile dùng hostname nội bộ Docker như discovery-server.

Cách vận hành/kiểm thử

- Chạy trong Compose trước các service nghiệp vụ để service đăng ký được lên registry.
- Kiểm tra UI tại http://localhost:8761 khi debug local.
- Các container phải gọi discovery-server:8080, không gọi localhost:8761.

Điểm review khi bảo vệ

- Điểm mạnh: module nhỏ, dễ kiểm soát, đúng vai trò hạ tầng.
- Điểm đã gia cố: prod profile dùng hostname discovery-server và discovery chỉ nằm trên internal network.
- Khi bảo vệ cần nhấn mạnh Eureka giúp gateway không phụ thuộc địa chỉ vật lý của từng service.

5.2. Module lifebalance-common

Vai trò	Shared API/Error/Web Utilities
Entrypoint	Không có application main
Số file Java/test	10 Java, 2 test
Dependency chính	Spring Boot, Jackson, validation support
Endpoint/contract chính	Không expose endpoint riêng
Port/vận hành	Không chạy độc lập

Công dụng

- Chuẩn hóa response API, lỗi nghiệp vụ và một số tiện ích web dùng chung.
- Giảm lặp code giữa các service, nhất là phần ApiResponse, ApiError và exception handler.
- Tạo contract thống nhất để frontend xử lý success/error dễ hơn.

Cách thiết kế

- Là Maven module dạng thư viện, được các service khác import như dependency nội bộ.
- Chứa các lớp cross-cutting không phụ thuộc domain cụ thể.
- Không có database, không có controller nghiệp vụ, không cần port runtime.

Cách viết code trong module

- ApiResponse<T> bao gồm success, data, error, timestamp.
- ApiError gom code, message, details để lỗi trả về có cấu trúc.
- Các exception/error code nên ổn định để frontend và tài liệu API không bị vỡ contract.

Cách vận hành/kiểm thử

- Được build cùng parent Maven trước hoặc cùng lúc với các service phụ thuộc.

- Không deploy riêng; khi common thay đổi cần chạy lại test các service dùng common.
- Nếu đổi schema response cần xem lại gateway, frontend và integration test.

Điểm review khi bảo vệ

- Điểm mạnh: giảm trùng lặp và giữ format lỗi nhất quán.
- Rủi ro: nếu common phình to thành nơi chứa mọi thứ, coupling giữa module sẽ tăng.
- Khi bảo vệ cần trình bày common là shared contract, không phải nơi chứa nghiệp vụ.

5.3. Module lifebalance-security

Vai trò	Security Auto-Configuration
Entrypoint	Không có application main
Số file Java/test	13 Java, 9 test
Dependency chính	Spring Security, OAuth2 Resource Server, JWT, Keycloak integration
Endpoint/contract chính	Không expose endpoint riêng; cấu hình filter chain cho service sử dụng
Port/vận hành	Không chạy độc lập

Công dụng

- Đóng gói cấu hình bảo mật dùng chung cho gateway và các service.
- Validate JWT từ Keycloak, map user/role và chuẩn hóa phản hồi 401/403.
- Giảm nguy cơ mỗi service tự cấu hình security theo một kiểu khác nhau.

Cách thiết kế

- Dùng auto-configuration để service chỉ cần thêm dependency là nhận SecurityFilterChain chung.
- Cho phép public các endpoint health/info/prometheus/swagger cần thiết cho vận hành.
- Tách method security để các service có thể dùng @PreAuthorize ở lớp service/controller.

Cách viết code trong module

- JwtAudienceValidator kiểm tra audience theo client-id.
- KeycloakUserMappingFilter map thông tin người dùng từ JWT vào context.
- AuthenticationEntryPoint và AccessDeniedHandler trả lỗi theo ApiError thay vì lỗi HTML mặc định.

Cách vận hành/kiểm thử

- Cần biến môi trường issuer-uri/client-id đúng với Keycloak realm.
- Khi chạy Docker, service phải gọi Keycloak bằng DNS nội bộ keycloak:8080.
- Nếu endpoint public bị 401, cần kiểm tra thứ tự matcher trong SecurityFilterChain.

Điểm review khi bảo vệ

- Điểm mạnh: security tập trung, dễ review hơn so với cấu hình rải rác.
- Rủi ro: thay đổi trong module này có blast radius lớn vì ảnh hưởng nhiều service.
- Khi bảo vệ cần nêu rõ JWT được validate ở gateway và service, không tin dữ liệu client gửi lên.

5.4. Module gateway

Vai trò	API Gateway
Entrypoint	com.lifebalance.gateway.GatewayApplication
Số file Java/test	1 Java, 3 test
Dependency chính	Spring Cloud Gateway MVC, Eureka Client, OAuth2 Resource Server, Actuator
Endpoint/contract chính	Route các prefix /api/... tới lb://IDENTITY-SERVICE, TASK-SERVICE, RESOURCE-CAPITAL-SERVICE, ...
Port/vận hành	8080 public endpoint

Công dụng

- Là cửa ngõ duy nhất để frontend/mobile gọi vào backend.
- Ẩn địa chỉ service thật, định tuyến request theo path và tên service trong Eureka.
- Tập trung các concern biên hệ thống như authentication, observability và route management.

Cách thiết kế

- Không chứa nghiệp vụ, chỉ có GatewayApplication và cấu hình route trong application.yaml.
- Dùng URI dạng lb://SERVICE-NAME để Spring Cloud LoadBalancer lấy instance từ Eureka.
- Route chính: identity, task, resource-capital, timeline, finance, notification, ai và analytics.

Cách viết code trong module

- Giữ route rõ ràng theo bounded context: `/api/tasks/**` cho task, `/api/finance/**` cho finance.
- Khi thêm endpoint mới ở service, cần cập nhật route gateway và test routing tương ứng.
- Không nên hard-code host/port service trong Java code; cấu hình nằm trong YAML/env.

Cách vận hành/kiểm thử

- Chạy trên port 8080, là service public duy nhất nằm ở edge network.
- Phụ thuộc discovery-server để resolve `lb://SERVICE-NAME`.
- Health endpoint: `/actuator/health/readiness` và `/actuator/health/liveness`.

Điểm review khi bảo vệ

- Điểm mạnh: gateway mỏng, đúng vai trò định tuyến, dễ giải thích khi bảo vệ.
- Điểm cần review: test Prometheus từng nhận 401 trong khi kỳ vọng 200; cần giữ thống nhất rule public actuator.
- Điểm cần review: test cũ dùng `/api/v1/capitals/summary` trong khi route/controller hiện là `/api/v1/capital/summary`.

5.5. Module identity-service

Vai trò	Identity, Authorization, Administration Support
Entrypoint	<code>com.lifebalance.identity.IdentityServiceApplication</code>
Số file Java/test	179 Java, 43 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, OAuth2/Keycloak admin client
Endpoint/contract chính	<code>/auth</code> , <code>/api/auth</code> , <code>/users</code> , <code>/api/users</code> , <code>/roles</code> , <code>/permissions</code> , <code>/audit-logs</code> , <code>/administration-support</code>
Port/vận hành	8091 khi debug trực tiếp

Công dụng

- Quản lý người dùng, vai trò, quyền, thông tin hồ sơ và audit log.
- Làm trung tâm danh tính để các module khác gắn dữ liệu theo `userId`.
- Cung cấp các API hỗ trợ quản trị như cấu hình, thông báo hệ thống, report hỗ trợ.

Cách thiết kế

- Tách controller theo nhóm `auth/user/role/permission/audit/administration-support`.
- Service xử lý nghiệp vụ và tích hợp Keycloak, repository chịu trách nhiệm truy vấn dữ liệu.
- Có package `audit/config/security` để xử lý các concern riêng của định danh.

Cách viết code trong module

- Controller nhận request/DTO, validate input và trả `ApiResponse`.
- Service nên bao transaction boundary, mapping entity/DTO và gọi Keycloak admin khi cần.
- Repository chỉ chứa truy vấn persistence, không chứa quyết định nghiệp vụ.

Cách vận hành/kiểm thử

- Cần PostgreSQL, Keycloak issuer/admin client và Eureka.
- Chạy migration Flyway trước khi service sẵn sàng.
- Khi demo cần có realm/client/user mẫu trong Keycloak hoặc seed dữ liệu tương ứng.

Điểm review khi bảo vệ

- Điểm mạnh: test nhiều nhất trong backend, phù hợp vì identity là module rủi ro cao.
- Cần review kỹ phân quyền role/permission, audit log và đồng bộ với Keycloak.
- Khi bảo vệ cần nhấn mạnh authentication do Keycloak/JWT đảm nhiệm, service quản lý dữ liệu mở rộng.

5.6. Module task-service

Vai trò	Task Management
Entrypoint	<code>com.lifebalance.task.TaskServiceApplication</code>
Số file Java/test	89 Java, 19 test
Dependency chính	Spring Web, JPA, PostgreSQL, Validation, Flyway, security library
Endpoint/contract chính	<code>/api/tasks</code> , <code>/api/categories</code> , <code>/api/tags</code> , <code>/tags</code> , <code>/api/tasks/reminders</code> , <code>/api/tasks/recurring-rules</code> , <code>/api/tasks/timeline</code>
Port/vận hành	8092 khi debug trực tiếp

Công dụng

- Quản lý nhiệm vụ, category, tag, reminder, recurring rule và lịch sử thay đổi task.
- Là module trung tâm để người dùng lập kế hoạch công việc.
- Cung cấp dữ liệu đầu vào cho timeline, analytics và AI recommendation.

Cách thiết kế

- Domain gồm Task, Category, Tag, Reminder, RecurringRule và History.
- Controller tách theo tài nguyên; service xử lý workflow như tạo, cập nhật, hoàn thành, nhắc việc.
- Có integration package để giao tiếp với timeline hoặc service khác khi cần.

Cách viết code trong module

- DTO/request cần validate deadline, recurrence rule và trạng thái task.
- Business rule như transition trạng thái nên nằm trong service/domain, không đặt trong controller.
- History nên được ghi khi thay đổi trường quan trọng để phục vụ audit và analytics.

Cách vận hành/kiểm thử

- Cần DB riêng, Eureka, Keycloak issuer và migration dữ liệu.
- Các API đi qua gateway bằng prefix `/api/tasks/**`.
- Khi demo nên chuẩn bị dữ liệu category/tag/task mẫu để thể hiện luồng nghiệp vụ.

Điểm review khi bảo vệ

- Điểm mạnh: bounded context rõ, số lượng test tương đối tốt.
- Cần review xử lý recurring/reminder để tránh sinh trùng task hoặc sai múi giờ.
- Cần kiểm tra quyền sở hữu userId để người dùng không truy cập task của người khác.

5.7. Module timeline-service

Vai trò	Timeline and Scheduling
Entrypoint	com.lifebalance.timeline.TimelineServiceApplication
Số file Java/test	40 Java, 4 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, validation/security library
Endpoint/contract chính	/api/timeline, /api/timeline/placements, /api/timeline/day, /api/timeline/week, /api/timeline/availability, /api/timeline/tasks, /api/timeline/history
Port/vận hành	8096 khi debug trực tiếp

Công dụng

- Sắp xếp task lên dòng thời gian theo ngày/tuần.
- Kiểm tra slot trống, conflict và khả năng đặt lịch.
- Lưu lịch sử placement để người dùng theo dõi thay đổi kế hoạch.

Cách thiết kế

- Domain tập trung quanh placement, availability, history và task projection.
- Controller cung cấp API day/week view, availability và quản lý placement.
- Validation bảo vệ các rule về thời gian bắt đầu, kết thúc và xung đột lịch.

Cách viết code trong module

- Nên dùng kiểu thời gian rõ ràng, thống nhất timezone và không trộn string date thủ công.
- Logic conflict/overlap nên có test riêng vì dễ phát sinh lỗi biên.
- Khi gọi task-service cần tránh coupling mạnh; chỉ giữ dữ liệu cần thiết cho timeline.

Cách vận hành/kiểm thử

- Direct debug port 8096; qua gateway dùng `/api/timeline/**`.
- Cần dữ liệu task hợp lệ trước khi demo placement.
- Nên chuẩn bị case lịch trống, lịch xung đột và lịch theo tuần để bảo vệ.

Điểm review khi bảo vệ

- Điểm mạnh: module tách riêng scheduling nên task-service không bị phình logic thời gian.
- Cần bổ sung test conflict/timezone nếu mở rộng scheduling.
- Khi bảo vệ cần giải thích vì sao timeline là bounded context riêng thay vì đặt trong task-service.

5.8. Module resource-capital-service

Vai trò	Resource Capital Management
Entrypoint	com.lifebalance.resourcecapital.ResourceCapitalServiceApplication
Số file Java/test	145 Java, 47 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, security, OpenFeign/WebClient nếu tích hợp
Endpoint/contract chính	/api/v1/capital, /api/v1/capital-cycles, /api/v1/capital-adjustments, /api/v1/capital-allocations
Port/vận hành	Không publish port debug riêng trong danh sách mặc định; đi qua gateway

Công dụng

- Quản lý vốn nguồn lực cá nhân như năng lượng, thời gian và capacity theo chu kỳ.
- Theo dõi điều chỉnh, phân bổ và summary vốn nguồn lực.
- Hỗ trợ LifeBalance nhìn người dùng không chỉ theo task mà còn theo năng lực thực thi.

Cách thiết kế

- Domain chia theo capital, cycle, adjustment và allocation.
- Controller route dùng số ít /api/v1/capital để đồng bộ với gateway.
- Có infrastructure/integration để tách phần giao tiếp ngoài domain thuần.

Cách viết code trong module

- Các phép cộng/trừ resource cần bảo vệ invariant như không âm, không vượt capacity.
- DTO cần validate unit, amount, cycle và khoảng thời gian.
- Service nên giữ transaction khi tạo adjustment/allocation để dữ liệu không lệch.

Cách vận hành/kiểm thử

- Cần DB riêng, Eureka và security issuer.
- Khi demo nên có dữ liệu cycle, adjustment và allocation để trình bày đầy đủ.
- Gateway route đúng là /api/v1/capital/**, không phải /api/v1/capitals/**.

Điểm review khi bảo vệ

- Điểm mạnh: số lượng test cao, phù hợp với module có nhiều rule nghiệp vụ.
- Cần review kỹ transaction và concurrency nếu nhiều request cập nhật capital cùng lúc.
- Khi bảo vệ nên nhấn mạnh đây là điểm khác biệt nghiệp vụ của đề tài LifeBalance.

5.9. Module finance-service

Vai trò	Finance Management
Entrypoint	com.lifebalance.finance.FinanceServiceApplication
Số file Java/test	74 Java, 5 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, security library
Endpoint/contract chính	/api/finance/accounts, /api/finance/categories, /api/budgets, /api/transactions, /api/finance/reports, /api/finance/recurring-transactions, /api/finance/history
Port/vận hành	8093 khi debug trực tiếp

Công dụng

- Quản lý tài khoản tài chính, danh mục, ngân sách, giao dịch và recurring transaction.
- Tạo report tài chính và lưu lịch sử thay đổi.
- Kết nối góc nhìn tài chính với cân bằng cuộc sống của người dùng.

Cách thiết kế

- Domain gồm Account, Budget, Category, Transaction, RecurringTransaction và Report.
- Controller tách theo tài nguyên, giúp route dễ tìm và dễ test.
- Repository/JPA quản lý persistence, service xử lý tính toán số dư/ngân sách.

Cách viết code trong module

- Luôn dùng BigDecimal cho tiền, không dùng float/double.
- Transaction create/update/delete nên cập nhật hoặc kiểm tra số dư trong cùng transaction.
- DTO cần validate amount, currency, date range và ownership user.

Cách vận hành/kiểm thử

- Direct debug port 8093; qua gateway dùng `/api/finance/**`, `/api/budgets/**`, `/api/transactions/**`.
- Cần migration DB và dữ liệu category/account mẫu khi demo.
- Nếu thêm integration ngân hàng bên ngoài cần quản lý secret và retry/fallback.

Điểm review khi bảo vệ

- Điểm mạnh: bounded context tài chính rõ ràng.
- Cần bổ sung test cho rule ngân sách, recurring transaction và report.
- Cần review bảo mật vì dữ liệu tài chính có tính nhạy cảm cao.

5.10. Module notification-service

Vai trò	Notification and Preference Management
Entrypoint	com.lifebalance.notification.NotificationServiceApplication
Số file Java/test	57 Java, 4 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, security library
Endpoint/contract chính	/api/notifications, /api/notifications/templates, /api/notifications/preferences, /api/notifications/delivery, /api/notifications/history
Port/vận hành	8094 khi debug trực tiếp

Công dụng

- Quản lý thông báo, template, preference, delivery state và lịch sử gửi.
- Hỗ trợ reminder task, cảnh báo tài chính hoặc thông báo hệ thống.
- Tách kênh giao tiếp với người dùng ra khỏi các service nghiệp vụ lõi.

Cách thiết kế

- Controller theo template/preference/delivery/history để mỗi API có trách nhiệm riêng.
- Service xử lý tạo notification, lựa chọn kênh và cập nhật trạng thái gửi.
- Repository lưu template, preference và lịch sử để truy vết.

Cách viết code trong module

- Không để controller tự ghép nội dung thông báo; nên thông qua template/service.
- Preference của user phải được kiểm tra trước khi gửi.
- Delivery retry cần idempotent để tránh gửi lặp khi có lỗi mạng.

Cách vận hành/kiểm thử

- Direct debug port 8094; qua gateway dùng `/api/notifications/**`.
- Có thể tích hợp RabbitMQ/Redis khi bật profile messaging/cache.
- Khi demo nên tạo preference và template mẫu trước.

Điểm review khi bảo vệ

- Điểm mạnh: notification tách riêng nên task/finance không phải gánh logic gửi.
- Cần review retry, idempotency và audit trail khi nâng cấp gửi email/push thật.
- Cần test thêm cho preference và delivery state.

5.11. Module analytics-service

Vai trò	Tracking, Evaluation, Reporting
Entrypoint	com.lifebalance.analytics.AnalyticsServiceApplication
Số file Java/test	61 Java, 8 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, security library
Endpoint/contract chính	/api/analytics/dashboard, /api/analytics/reports, /api/analytics/tracking-evaluation, /api/analytics/evaluations, /api/analytics/actual-records, /api/analytics/history
Port/vận hành	8097 khi debug trực tiếp

Công dụng

- Tổng hợp dashboard, report, tracking, evaluation và actual record.
- So sánh kế hoạch với thực tế để đánh giá tiến độ cân bằng cuộc sống.
- Cung cấp dữ liệu đầu ra cho báo cáo và insight AI.

Cách thiết kế

- Domain tách evaluation, tracking record, report và history.
- Controller chia theo dashboard/report/tracking-evaluation/evaluations/actual-records.

- Service tính toán chỉ số, aggregate dữ liệu và chuẩn bị response cho UI.

Cách viết code trong module

- Tách query/read model nếu report ngày càng phức tạp.
- Không nên để controller tự tính toán metric; controller chỉ nhận input và trả output.
- Cần test các công thức tổng hợp để tránh sai số khi dữ liệu lớn.

Cách vận hành/kiểm thử

- Direct debug port 8097; qua gateway dùng `/api/analytics/**`.
- Cần dữ liệu task/timeline/finance/resource-capital mẫu để dashboard có ý nghĩa.
- Nếu report nặng, có thể bổ sung cache hoặc job nền.

Điểm review khi bảo vệ

- Điểm mạnh: tracking/evaluation được tách khỏi module tạo dữ liệu.
- Cần review hiệu năng truy vấn dashboard/report khi số bản ghi tăng.
- Khi bảo vệ cần nói rõ analytics là lớp đọc/tổng hợp, không thay đổi nghiệp vụ lõi.

5.12. Module ai-service

Vai trò	AI Recommendations and Insights
Entrypoint	com.lifebalance.ai.AiServiceApplication
Số file Java/test	55 Java, 5 test
Dependency chính	Spring Web, JPA, PostgreSQL, Flyway, security library
Endpoint/contract chính	/api/ai/recommendations, /api/ai/insights, /api/ai/conversations, /api/ai/history
Port/vận hành	8095 khi debug trực tiếp

Công dụng

- Quản lý recommendation, insight, conversation và history AI.
- Hỗ trợ người dùng ra quyết định bằng gợi ý dựa trên dữ liệu task/timeline/finance/resource.
- Tách AI ra service riêng để có thể thay provider/model mà không ảnh hưởng service lõi.

Cách thiết kế

- Domain gồm recommendation, insight, conversation, message và history.
- Controller tách recommendation/insight/conversation/history/status.
- Service là nơi nên đặt adapter tới provider AI hoặc rule engine nội bộ.

Cách viết code trong module

- Nên có lớp adapter/provider để đổi model AI mà ít ảnh hưởng controller.
- Prompt, input context và output cần được validate để tránh lộ dữ liệu nhạy cảm.
- Apply/dismiss recommendation nên ghi trạng thái để analytics đo tác động.

Cách vận hành/kiểm thử

- Direct debug port 8095; qua gateway dùng `/api/ai/**`.
- Cần DB AI, Keycloak issuer và Eureka.
- Nếu tích hợp external AI provider cần quản lý secret/env riêng và fallback khi provider lỗi.

Điểm review khi bảo vệ

- Điểm mạnh: AI được tách riêng nên không làm nhiễm service nghiệp vụ core.
- Cần review privacy vì AI có thể xử lý dữ liệu cá nhân.
- Cần review rate limit, prompt logging và error handling trước demo nâng cao.

6. Checklist review và bảo vệ đồ án

Chủ đề	Câu hỏi có thể bị hỏi	Cách trả lời ngắn gọn
Kiến trúc	Vì sao tách nhiều service?	Để tách bounded context, scale/test độc lập và tránh một monolith quá lớn.
Gateway	Gateway làm gì?	Là cửa vào API, route lb://SERVICE-NAME qua Eureka và áp dụng security/observability ở biên.
Discovery	Eureka có cần thiết không?	Cần khi service chạy container/port động; gateway không phải hard-code địa chỉ service.
Security	JWT được validate ở đâu?	lifebalance-security cấu hình OAuth2 Resource Server, validate issuer/audience và trả lỗi 401/403 chuẩn.
Database	Mỗi service có DB riêng không?	Compose tạo DB/schema riêng theo service, migration Flyway riêng để giảm coupling dữ liệu.
Code	Controller có chứa nghiệp vụ không?	Không. Controller chỉ nhận request/response; service chứa rule nghiệp vụ.
Testing	Module nào test nhiều nhất?	Identity và resource-capital có nhiều test vì rủi ro bảo mật/rule nghiệp vụ cao.
Monitoring	Quan sát hệ thống thế nào?	Actuator health/readiness/liveness và Prometheus/Grafana khi bật monitoring profile.

7. Rủi ro, tác động và khuyến nghị

Mức độ	Vấn đề	Tác động	Khuyến nghị
Cao	Sai cấu hình security public/private endpoint.	Health/Prometheus có thể bị 401 hoặc endpoint nhạy cảm bị mở nhầm.	Review SecurityFilterChain, test actuator và API protected/public.
Cao	Không kiểm tra owner/userId trong service.	Người dùng có thể đọc/sửa dữ liệu của người khác.	Bổ sung authorization ở service layer và test access control.
Trung bình	Gateway route không khớp controller path.	Frontend gọi API bị 404 hoặc route sai service.	Mỗi endpoint mới phải có test route và soát application.yaml gateway.
Trung bình	Dùng localhost trong container.	Service không kết nối được DB/Keycloak/Eureka khi chạy Docker.	Dùng Docker DNS nội bộ: postgres, keycloak, discovery-server.
Trung bình	Thiếu test timezone/recurring/concurrency.	Timeline/task/capital có thể sai dữ liệu biên.	Bổ sung test cho rule thời gian, recurring và cập nhật đồng thời.
Thấp	Common module phình to.	Tăng coupling giữa service.	Chỉ đưa contract/cross-cutting thật sự dùng chung vào common.

Các điểm đã ghi nhận trong quá trình review

- discovery-server đã được gia cố theo hướng an toàn hơn cho Docker: hostname prod là discovery-server và chỉ dùng internal network.
- gateway cần giữ test Prometheus thống nhất với rule public /actuator/prometheus; nếu endpoint dùng cho Prometheus scrape thì không nên bị 401.
- gateway/test cần thống nhất path singular/plural của resource-capital: code hiện dùng /api/v1/capital/**.
- Các module nghiệp vụ cần tiếp tục ưu tiên test authorization, ownership, validation và transaction.

8. Nguồn tham chiếu trong repo

- pom.xml parent: danh sách 12 module, Java 21, Spring Boot 3.5.x, Spring Cloud 2025.0.x.
- gateway/src/main/resources/application.yaml: cấu hình route lb://SERVICE-NAME.
- discovery-server/src/main/resources/application*.yaml: cấu hình Eureka theo profile.
- lifebalance-security/src/main/java/.../LifebalanceSecurityAutoConfiguration.java: SecurityFilterChain dùng chung.
- compose.yaml, compose.prod.yaml, DOCKER.md, RUN.md: cách vận hành Docker, port, healthcheck và monitoring.
- src/main/java/**/controller/*.java: endpoint chính của từng service.
- src/test/java và src/test/resources: test coverage và cấu hình test module.